

TD1 : Opérateurs ensemblistes, LDD et LCT — Corrigé ccb79a8

Rosine Cicchetti - Lotfi Lakhal - Mickaël Martin Nevot



Cette œuvre de Rosine Cicchetti est mise à disposition sous licence Creative Commons
Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : www.mickael-martin-nevot.com

1 Rappel : Prise en main d'un éditeur Oracle

Utilisez une des deux solutions ci-dessous.

1.1 Oracle Live SQL

Vous pouvez utiliser directement, sans installation ou configuration, l'interpréteur en ligne :
<https://livesql.oracle.com>.

1.2 Oracle Cloud Free Tier

Mettez en place une solution d'hébergement en ligne d'un SGBD Oracle. Vous pouvez pour cela
consulter le document **Vade-Mecum mise en place d'un hébergement Oracle Cloud Free Tier**.

Ajouter ensuite une base de données nommée **zenetude-bd**.

2 Rappel : Généralités

Voici la convention de nommage proposé dans cet enseignement :

- **mots clefs** : en lettre capitales (*upper case*) ;
- **relation** : première lettre de chaque mot en capitale (*pascal case*) ;
- **attributs** : premier mot en minuscule et première lettre de chaque mot suivant en capitale (*camel case*) ;
- **domaine** : en lettre capitales (*upper case*) au format D_XXX¹.

Pour rappel, les valeurs saisies dans une base de données, comme les chaînes de caractères, sont sensibles à la casse et, par convention, sont saisies **en lettres capitales** (*upper case*), sans diacritique, dans le cadre de cet enseignement.

1. Les domaines identiques sont surlignés avec la même couleur.

3 Rappel : Base de données exemple

La base de données exemple, Voyage, utilisée dans les documents de travaux dirigés de cet enseignement, permet à un réseau d’agences de voyage de gérer les clients, les voyages ainsi que leurs options, la planification des voyages et les réservations des clients.

3.1 Dictionnaire de données

La base de données Voyage a été élaborée à partir du dictionnaire des données suivant² :

Table 1 – Dictionnaire des données de la base de données exemple

Attribut	Description	Domaine	Remarques
idV	Identifiant d’un voyage	D_IDV : NUMBER(6,0)	Valeurs uniques
villeArr	Ville d’arrivée d’un voyage	D_VILLE : VARCHAR2(20)	
paysArr	Pays d’arrivée d’un voyage	D_PAYS : VARCHAR2(20)	
villeDep	Ville de départ d’un voyage	D_VILLE : VARCHAR2(20)	
hotel	Nom d’un hôtel	D_HOTEL : VARCHAR2(20)	
nbEtoiles	Nombre d’étoiles d’un hôtel	D_NBETOILE : NUMBER(1,0)	
duree	Nombre de jours d’un voyage	D_DUREE : NUMBER(2,0)	
dateDep	Date de départ	D_DATE : DATE	
tarif	Prix unitaire du voyage	D_TARIF : NUMBER(6,2)	
numCl	Numéro d’un client	D_NUMCL : NUMBER(6,0)	Valeurs uniques
nom	Nom d’un client	D_NOM : VARCHAR2(25)	
prenom	Prénom d’un client	D_PRENOM : VARCHAR2(20)	
adresse	Adresse d’un client	D_ADRESSE : VARCHAR2(40)	
cp	Code postal d’un client	D_CP : VARCHAR2(5)	
ville	Ville de résidence d’un client	D_VILLE : VARCHAR2(20)	
categorie	Catégorie d’un client	D_CATEGORIE : VARCHAR2(15)	
nbPers	Nombre de personnes	D_NBPERS : NUMBER(2,0)	
dateRes	Date de réservation	D_DATE : DATE	
code	Code d’une option	D_CODE : NUMBER(3,0)	Valeurs uniques
libelle	Libellé d’une option	D_LIBELLE : VARCHAR2(20)	Valeurs uniques

2. Les types syntaxiques, utilisés pour la description des domaines, sont disponibles dans Oracle.

Attribut	Description	Domaine	Remarques
prix	Prix d'une option	D_TARIF : NUMBER(6,2)	

Remarques

Chaque voyage a une destination composée d'une ville et d'un pays d'arrivée, une ville de départ, le nom de l'hôtel où sont accueillis les clients, son nombre d'étoiles et la durée du voyage en jour.

Pour chaque voyage, plusieurs départs sont planifiés, généralement longtemps à l'avance. Le tarif unitaire, pour une personne, varie selon la période.

Les clients sont identifiés par un numéro et ont différentes caractéristiques dont leur catégorie.

Les clients effectuent des réservations pour des voyages planifiés. Le nombre de personnes et la date de réservation sont conservés pour chaque réservation.

Enfin, des options peuvent être proposées pour les voyages (visite guidée, safari découverte, safari photo, etc.). Ces options peuvent être gratuites pour un voyage ; elles sont alors incluses dans le tarif.

Nous considérons qu'il n'y a pas deux personnes homonymes ayant le même prénom et le même nom.

3.2 MCD

Le MCD (voir figure 1) modélise quatre entités : **Voyage** (les séjours proposés par le réseau d'agences), **Client**, **OptionV** (les options proposées pour les voyages) et **Planning** (les départs planifiés). L'association identifiante **A** relie un voyage à ses plannings : chaque planning est identifié par sa date de départ au sein d'un voyage donné et porte le tarif unitaire. L'association **Réservation** relie un planning à un client et porte le nombre de personnes ainsi que la date de réservation. Enfin, l'association **Carac** relie un voyage à une option et porte le prix de cette option pour ce voyage.

3.3 Schéma relationnel

Les clefs primaires sont soulignées et les clefs étrangères en *italique suivies d'un #*.

Le schéma relationnel de la base de données exemple est présenté ci-après :

Voyage (idV, villeArr, paysArr, villeDep, hotel, nbEtoiles, duree)

Planning (*idV#*, dateDep, tarif)

Client (numCl, nom, prenom, adresse, cp, ville, categorie)

Reservation (*numCl#*, *idV#*, *dateDep#*, nbPers, dateRes)

OptionV (code, libelle)

Carac (*idV#*, *code#*, prix)

Remarques

La clef étrangère composite (*idV*, *dateDep*) dans **Reservation** fait référence à la clef primaire composite de **Planning**.



Figure 1 – Modèle conceptuel des données (MCD)

4 Requêtes avec SQL

Formulez, en SQL, sur la base de données exemple, les requêtes d'interrogation suivantes.

4.1 Rappel : expression des jointures

Quand cela est possible, formulez les requêtes suivantes de trois manières différentes.

Q1 Donnez les dates de départ, villes d'arrivée et tarifs des voyages à destination du Maroc.
3 attributs, 29 tuples

Version algébrique.

```

SELECT DISTINCT dateDep, villeArr, tarif
FROM Voyage V
    INNER JOIN Planning P
      ON V.idV = P.idV
WHERE paysArr = 'MAROC';
  
```

Remarques

la version imbriquée ne peut pas être effectuée utilement car dateDep et tarif sont des attributs de Planning et villeArr est un attribut de Voyage.

Version prédicative.

```
SELECT DISTINCT dateDep, tarif, villeArr
FROM Voyage V, Planning P
WHERE
    V.idV = P.idV
    AND paysArr = 'MAROC';
```

Q2 Donnez les dates de départ, villes d'arrivée et pays d'arrivée des voyages réservés des clients ne résidant ni à Paris ni à Marseille. *3 attributs, 2 tuples*

Version algébrique.

```
SELECT DISTINCT dateDep, villeArr, paysArr
FROM Voyage V
    INNER JOIN Reservation R
        ON V.idV = R.idV
    INNER JOIN Client C
        ON R.numCl = C.numCl
WHERE ville NOT IN ('PARIS', 'MARSEILLE');
```

Version imbriquée.

```
SELECT DISTINCT dateDep, villeArr, paysArr
FROM Voyage V
    INNER JOIN Reservation R
        ON
            V.idV = R.idV
            AND numCl IN (
                SELECT numCl
                FROM Client
                WHERE ville NOT IN ('PARIS', 'MARSEILLE')
            );
```

Remarques

la version imbriquée ne peut pas être effectuée parfaitement car dateDep est un attribut de Reservation et dateDep et tarif sont des attributs de Voyage.

Version prédicative.

```
SELECT DISTINCT dateDep, villeArr, paysArr
FROM Voyage V, Reservation R, Client C
WHERE
    V.idV = R.idV
    AND R.numCl = C.numCl
    AND ville NOT IN ('PARIS', 'MARSEILLE');
```

Q3 Quels sont les libellés des options gratuites proposées pour les voyages réservés par le client Nicolas Barbier ? *1 attribut, 3 tuples*

Version algébrique.

```

SELECT DISTINCT libelle
FROM OptionV O
    INNER JOIN Carac Ca
        ON O.code = Ca.code
    INNER JOIN Reservation R
        ON Ca.idV = R.idV
    INNER JOIN Client C
        ON R.numCl = C.numCl
WHERE
    prix IS NULL
    AND prenom = 'NICOLAS'
    AND nom = 'BARBIER';

```

Version imbriquée.

```

SELECT libelle
FROM OptionV
WHERE code IN (
    SELECT code
    FROM Carac
    WHERE
        (prix = 0 OR prix IS NULL)
        AND idV IN (
            SELECT idV
            FROM Reservation
            WHERE numCl IN (
                SELECT numCl
                FROM Client
                WHERE
                    prenom = 'NICOLAS'
                    AND nom = 'BARBIER'
            )
        )
);

```

Version prédicative.

```

SELECT DISTINCT libelle
FROM OptionV O, Carac Ca, Reservation R, Client C
WHERE
    O.code = Ca.code
    AND Ca.idV = R.idV
    AND R.numCl = C.numCl
    AND nom = 'BARBIER'
    AND prenom = 'NICOLAS'
    AND prix IS NULL;

```

Q4 Quels sont les noms, prénoms et villes de résidence des clients ayant réservé un voyage à destination d'Istanbul partant de leur ville de résidence ? *3 attributs, 5 tuples*

Version algébrique.

```

SELECT DISTINCT nom, prenom, ville
FROM Client C
    INNER JOIN Reservation R
        ON C.numCl = R.numCl
    INNER JOIN Voyage V
        ON
            R.idV = V.idV
            AND villeDep = ville
WHERE villeArr = 'ISTANBUL';

```

Version imbriquée.

```

SELECT nom, prenom, ville
FROM Client
WHERE (numCl, ville) IN (
    SELECT numCl, villeDep
    FROM Reservation R
        INNER JOIN Voyage V
            ON
                R.idV = V.idV
                AND villeArr = 'ISTANBUL'
);

```

Remarques

la version imbriquée ne peut pas être effectuée parfaitement car numCl est un attribut de Reservation et villeDep un attribut de Voyage.

Version prédicative.

```

SELECT DISTINCT nom, prenom, ville
FROM Voyage V, Reservation R, Client C
WHERE
    V.idV = R.idV
    AND R.numCl = C.numCl
    AND villeDep = ville
    AND villeArr = 'ISTANBUL';

```

4.2 Utilisation des opérateurs ensemblistes et équivalences

Formulez les requêtes suivantes en faisant appel aux opérateurs ensemblistes.

Q5 Quelles sont les villes de départ d'un voyage dans lesquelles résident des clients ? *1 attribut, 4 tuples*

```

SELECT villeDep
FROM Voyage
INTERSECT
SELECT ville
FROM Client;

```

Q6 Quels sont les libellés des options communes aux voyages d'identifiants 354 et 952? *1 attribut, 3 tuples*

```
SELECT libelle
FROM OptionV
WHERE code IN (
    SELECT code
    FROM Carac
    WHERE idV = 354
    INTERSECT
    SELECT code
    FROM Carac
    WHERE idV = 952
);
```

Q7 Donnez les identifiants, villes d'arrivée et pays d'arrivée des voyages pour lesquels il n'y a aucune réservation. *3 attributs, 14 tuples*

```
SELECT idV, villeArr, paysArr
FROM Voyage
WHERE idV IN (
    SELECT idV
    FROM Voyage
    EXCEPT
    SELECT idV
    FROM Reservation
);
```

Q8 Quels sont les libellés des options gratuites pour le voyage d'identifiant 354 et ceux des options payantes pour le voyage d'identifiant 952? *1 attribut, 5 tuples*

```
SELECT libelle
FROM OptionV
WHERE code IN (
    SELECT code
    FROM Carac
    WHERE
        (prix = 0 OR prix IS NULL)
        AND idV = 354
    UNION
    SELECT code
    FROM Carac
    WHERE
        (prix <> 0 OR prix IS NOT NULL)
        AND idV = 952
);
```

Formulez les requêtes suivantes en ne faisant pas appel aux opérateurs ensemblistes.

Q9 Quels sont les identifiants, villes d'arrivée et pays d'arrivée des voyages offrant à la fois les options de visite guidée et de piscine? *3 attributs, 1 tuple*

Version algébrique.

```
SELECT DISTINCT V.idV, villeArr, paysArr
FROM Voyage V
  INNER JOIN Carac C1
    ON V.idV = C1.idV
  INNER JOIN OptionV O1
    ON C1.code = O1.code
  INNER JOIN Carac C2
    ON V.idV = C2.idV
  INNER JOIN OptionV O2
    ON C2.code = O2.code
WHERE
  O1.libelle = 'VISITE GUIDEE'
  AND O2.libelle = 'PISCINE';
```

Version imbriquée.

```
SELECT idV, villeArr, paysArr
FROM Voyage
WHERE
  idV IN (
    SELECT idV
    FROM Carac
    WHERE code IN (
      SELECT code
      FROM OptionV
      WHERE libelle = 'PISCINE'
    )
  )
  AND idV IN (
    SELECT idV
    FROM Carac
    WHERE code IN (
      SELECT code
      FROM OptionV
      WHERE libelle = 'VISITE GUIDEE'
    )
  )
);
```

Version prédicative.

```
SELECT DISTINCT V.idV, villeArr, paysArr
FROM Voyage V, Carac C1, OptionV O1, Carac C2, OptionV O2
WHERE
  V.idV = C1.idV
  AND C1.code = O1.code
  AND V.idV = C2.idV
  AND C2.code = O2.code
  AND O1.libelle = 'VISITE GUIDEE'
  AND O2.libelle = 'PISCINE';
```

Q10 Donnez les noms et prénoms des clients qui n'ont aucune réservation. *2 attributs, 11 tuples*

V1.

```
SELECT nom, prenom
FROM Client
WHERE numCl NOT IN (
    SELECT numCl
    FROM Reservation
);
```

V2.

```
SELECT nom, prenom
FROM Client
WHERE numCl <> ALL ( -- noqa: LT01, LT06
    SELECT numCl
    FROM Reservation
);
```

V3.

```
SELECT C.nom, C.prenom
FROM Client C
WHERE NOT EXISTS (
    SELECT *
    FROM Reservation R
    WHERE R.numCl = C.numCl -- noqa: RF03
);
```

4.3 Création et modification d'attributs

Formulez les requêtes suivantes en pensant générer, avec la commande DESCRIBE, un affichage permettant de vérifier la validité des réponses et, à la fin de la séance, penser à annuler les modifications faites pour laisser la base de données dans l'état initial.

Q11 Ajoutez les attributs correspondant au tarif enfant et au nombre d'enfants.

```
ALTER TABLE Planning ADD (tarifEnf NUMBER(6, 2));
ALTER TABLE Reservation ADD (nbEnf NUMBER(2, 0));
DESCRIBE Planning;
DESCRIBE Reservation;
```

Q12 Doublez la taille possible du libellé d'une option.

```
ALTER TABLE OptionV MODIFY (libelle VARCHAR2(40));
DESCRIBE OptionV;
```

Q13 En se basant sur les données actuelles, définissez une contrainte de domaine pour les catégories. Vérifiez en essayant d'ajouter un client ne respectant pas cette contrainte.

```
ALTER TABLE Client ADD (CONSTRAINT ck_Client_categorie CHECK (categorie IN ('PRIVILEGIE', 'E
DESCRIBE Client;
```

Insertion invalide.

```
INSERT INTO Client (numCl, nom, prenom, adresse, cp, ville, categorie) VALUES (666, 'MARTIN
```

Remarques
cette insertion provoque une erreur.

Q14 En se basant sur les données actuelles, définissez une contrainte de domaine pour les nombres d'étoiles. Vérifiez en essayant d'ajouter un voyage ne respectant pas cette contrainte.

```
ALTER TABLE Voyage ADD (CONSTRAINT ck_Voyage_nbEtoiles CHECK (nbEtoiles BETWEEN 3 AND 5));
DESCRIBE Voyage;
```

Insertion invalide.

```
INSERT INTO Voyage (idV, villeArr, paysArr, villeDep, hotel, nbEtoiles, duree) VALUES (666,
```

Remarques
cette insertion provoque une erreur.

Q15 Créez une nouvelle relation *Capacite* permettant de connaître par hôtel et type de chambre *typeC*, pouvant être SIMPLE, DOUBLE, DOUBLE LUXE, SUITE, SUITE JUNIOR et SUITE PRESTIGE, le nombre de chambres *nbCh*.

```
DROP TABLE IF EXISTS Capacite PURGE;
CREATE TABLE Capacite (
    hotel VARCHAR2(20),
    typeC VARCHAR2(15),
    nbCh NUMBER(3, 0),
    CONSTRAINT pk_Capacite PRIMARY KEY(hotel, typeC),
    CONSTRAINT ck_Capacite_typeC CHECK(typeC IN ('SIMPLE', 'DOUBLE', 'DOUBLE LUXE', 'SUITE',
));
DESCRIBE Capacite;
```

4.4 Mises à jour des données

Formulez les requêtes suivantes en pensant générer, avec la commande SELECT ou DESCRIBE, un affichage permettant de vérifier la validité des réponses et, à la fin de la séance, penser à annuler les modifications faites pour laisser la base de données dans l'état initial.

Q16 Le client numéro 2103 réserve toujours avec ses deux enfants et le client Thomas Jarolim avec son enfant unique. *4 tuples*

```

ALTER TABLE Planning ADD (tarifEnf NUMBER(6, 2));
ALTER TABLE Reservation ADD (nbEnf NUMBER(2, 0));
UPDATE Reservation
SET nbEnf = 2
WHERE numCl = 2103;
UPDATE Reservation
SET nbEnf = 1
WHERE numCl IN (
    SELECT numCl
    FROM Client
    WHERE nom = 'JAROLIM'
    AND prenom = 'THOMAS'
);
SELECT C.numCl, nom, prenom, idV, dateDep, nbEnf
FROM Client C
    INNER JOIN Reservation R
    ON C.numCl = R.numCl
WHERE
    C.numCl = 2103
    OR (nom = 'JAROLIM' AND prenom = 'THOMAS');

```

Remarques

les commandes précédentes sont nécessaires si les altérations de table n'ont pas été faites précédemment. Elles ne doivent pas être exécutées dans le cas contraire.

Q17 Un tarif enfant est moitié prix du tarif correspondant. *80 tuples*

```

ALTER TABLE Planning ADD (tarifEnf NUMBER(6, 2));
UPDATE Planning SET tarifEnf = tarif / 2;
SELECT *
FROM Planning;

```

Remarques

la commande précédente est nécessaire si l'altération de table n'a pas été faite précédemment. Elle ne doit pas être exécutée dans le cas contraire.

Q18 Insérez les données suivantes. *12 tuples*

Extension de la relation **Capacite** de la base de données Voyage

hotel	typeC	nbCh
ANTIQUE	SIMPLE	10
ANTIQUE	DOUBLE	75
ANTIQUE	DOUBLE LUXE	12
ANTIQUE	SUITE	5
ELIAS BEACH	DOUBLE	83
ELIAS BEACH	SUITE	27
OLD BRIDGE	SIMPLE	25

hotel	typeC	nbCh
OLD BRIDGE	DOUBLE	75
SAFARI JAMBO	SIMPLE	32
SAFARI JAMBO	DOUBLE	100
TRANSATLANTIQUE	DOUBLE	200
BAMBURI	DOUBLE	150

```

DROP TABLE IF EXISTS Capacite PURGE;
CREATE TABLE Capacite (
    hotel VARCHAR2(20),
    typeC VARCHAR2(15),
    nbCh NUMBER(3, 0),
    CONSTRAINT pk_Capacite PRIMARY KEY(hotel, typeC),
    CONSTRAINT ck_Capacite_typeC CHECK(typeC IN ('SIMPLE', 'DOUBLE', 'DOUBLE LUXE', 'SUITE',
));
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('ANTIQUE', 'SIMPLE', 10);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('ANTIQUE', 'DOUBLE', 75);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('ANTIQUE', 'DOUBLE LUXE', 12);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('ANTIQUE', 'SUITE', 5);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('ELIAS BEACH', 'DOUBLE', 83);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('ELIAS BEACH', 'SUITE', 27);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('OLD BRIDGE', 'SIMPLE', 25);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('OLD BRIDGE', 'DOUBLE', 75);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('SAFARI JAMBO', 'SIMPLE', 32);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('SAFARI JAMBO', 'DOUBLE', 100);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('TRANSATLANTIQUE', 'DOUBLE', 200);
INSERT INTO Capacite (hotel, typeC, nbCh) VALUES ('BAMBURI', 'DOUBLE', 150);
SELECT *
FROM Capacite;

```

Remarques

les commandes précédentes sont nécessaires si la création de table n'a pas été faite précédemment. Il est inutile de les exécuter dans le cas contraire.

4.5 Archivage d'information et gestion de transactions

Le mécanisme des transactions est ici illustré à travers un exemple d'archivage d'information.

Dans un souci d'archivage des données, on désire régulièrement purger la relation **Reservation**, sans pour autant perdre les informations existantes.

Q19 Créez une nouvelle relation **AncienneReservation** permettant d'archiver toutes les réservations programmées passées, la date de réservation étant remplacée par la date d'archivage.

```

DROP TABLE IF EXISTS AncienneReservation PURGE;
CREATE TABLE AncienneReservation (
    numCl NUMBER(6, 0),
    idV NUMBER(6, 0),
    dateDep DATE,
    nbPers NUMBER(2, 0),

```

```

    dateArch DATE,
    nbEnf NUMBER(2, 0),
    CONSTRAINT pk_AncienneReservation PRIMARY KEY(numCl, idV, dateDep)
);
DESCRIBE AncienneReservation;

```

Q20 À l'aide d'une transaction, archivez les réservations antérieures à 2004. *AncienneReservation : 14 tuples, Reservation : 18 tuples*

```

ALTER TABLE Reservation ADD (nbEnf NUMBER(2, 0));
DROP TABLE IF EXISTS AncienneReservation PURGE;
CREATE TABLE AncienneReservation (
    numCl NUMBER(6, 0),
    idV NUMBER(6, 0),
    dateDep DATE,
    nbPers NUMBER(2, 0),
    dateArch DATE,
    nbEnf NUMBER(2, 0),
    CONSTRAINT pk_AncienneReservation PRIMARY KEY(numCl, idV, dateDep)
);
BEGIN
    INSERT INTO AncienneReservation
    SELECT numCl, idV, dateDep, nbPers, SYSDATE, nbEnf
    FROM Reservation
    WHERE dateRes < TO_DATE('2004-01-01', 'YYYY-MM-DD');
    DELETE FROM Reservation
    WHERE dateRes < TO_DATE('2004-01-01', 'YYYY-MM-DD');
END;
/
SELECT *
FROM AncienneReservation;
SELECT *
FROM Reservation;

```

Remarques

les commandes précédentes sont nécessaires si l'alteration et la création de table n'ont pas été faites précédemment. L'altération ne doit pas être exécutée, et il est inutile d'exécuter la création dans le cas contraire.